

Pipeline Vidéo USB Caméra → HDMI 720p

sur PYNQ-Z1 (Zynq-7020) — Vivado 2024.1

Projet PA2 — Documentation technique

Élément	Détail
Cursus	Master Informationstechnik / Embedded Systems
Établissement	FH Dortmund
Superviseur	Prof. Dr. Michael Karagounis
Carte cible	PYNQ-Z1 — Zynq XC7Z020-CLG400-1
Caméra	Logitech C270 (720p MJPG @ 30 fps)
Outil	Vivado 2024.1 / PYNQ

1. Objectif du projet

L'objectif est de construire un pipeline vidéo matériel complet sur la carte PYNQ-Z1 : capturer le flux d'une caméra USB, le transférer en mémoire DDR, puis le restituer sur un écran via la sortie HDMI en résolution 720p. Ce pipeline constitue le socle d'une étape ultérieure : l'insertion d'une IP d'inférence MobileNetV2 (détection binaire de personne) directement dans la chaîne de traitement, avec retour visuel à l'écran (bannière d'état et bordure colorée).

Ce document retrace l'architecture retenue, le déroulé du débogage (notamment la résolution d'un problème d'horloge pixel qui produisait un écran noir), le code final fonctionnel, et les points encore ouverts.

2. Architecture du pipeline

Le traitement est réparti entre le PS (Processing System, le cœur ARM) et la PL (Programmable Logic, le FPGA). Le PS gère la caméra USB via OpenCV/V4L2 et écrit les images dans un framebuffer contigu en DDR. La PL lit ce framebuffer par DMA et le convertit en signal HDMI.

```
Caméra USB C270 --(OpenCV / V4L2, MJPG)--> PS (ARM Cortex-A9)
      |
      v  (écriture du framebuffer)
DDR   (framebuffer contigu, alloué via pynq.allocate)
      |
      v  AXI VDMA -- canal MM2S (mémoire -> flux AXI-Stream)
AXI4-Stream
      |
      v
v_axi4s_vid_out <-- timing -- v_tc_0 (Video Timing Controller, 720p figé)
      |
      v
rgb2dvi  (encodage TMDS)
      |
      v
HDMI OUT --> Moniteur
```

Caption : chaîne complète du capteur à l'écran.

2.1 Rôle de chaque bloc IP

Bloc IP	Rôle	AXI-Lite ?
axi_vdma_0	Lit le framebuffer DDR, émet un flux AXI-Stream (MM2S)	Oui
v_tpg_0	Générateur de mire de test (validation)	Oui
v_axi4s_vid_out_0	Convertit AXI-Stream -> vidéo native, locké sur le VTC	Non
v_tc_0	Génère le timing 720p (sync, porches). Mode standalone	Non
rgb2dvi	Encode le RGB en TMDS pour le HDMI	Non
clk_wiz_1	Génère l'horloge pixel 74,25 MHz	Non

Point important : seuls les IP dotés d'une interface AXI-Lite apparaissent dans le dictionnaire `ip_dict` de PYNQ. Le `v_tc_0` étant configuré en mode standalone (timing figé dans le bitstream), il n'apparaît jamais dans `ip_dict` — c'est normal et attendu, ce n'est pas le signe d'une IP manquante. Les clés attendues sont : `axi_vdma_0`, `v_tpg_0`, `ps7`.

3. Le problème d'horloge pixel (écran noir)

Le symptôme initial était un écran noir complet, alors que le bitstream de base (base.bit) affichait correctement — ce qui prouvait que le câble, le moniteur et le matériel étaient bons. Le défaut venait donc du design personnalisé.

3.1 Cause racine

Pour un signal 720p60, l'horloge pixel doit valoir exactement **74,25 MHz**. Les trois blocs du chemin vidéo — `v_tc_0`, `v_axis_vid_out_0` et `rgb2dvi` — doivent tous être cadencés par cette **même** horloge. Le design initial laissait certains blocs à 100 MHz, et une tentative intermédiaire a même introduit deux horloges différentes, créant un croisement de domaines d'horloge (clock-domain crossing) qui cassait le timing.

Les erreurs de validation Vivado confirmaient le diagnostic :

```
FREQ_HZ does not match ... 74250000 vs 100000000
CLK_DOMAIN does not match clk_wiz_1_clk_out1 vs FCLK_CLK1
```

3.2 Solution adoptée

Une horloge unique pour tout le sous-système vidéo. Le `clk_wiz` a été configuré pour sortir 74,25 MHz (onglet Output Clocks), et toutes les horloges du chemin vidéo ont été reliées à cette unique source (`clk_wiz_1/clk_out1`). Après reconstruction du bitstream, la validation passait et l'affichage fonctionnait.

Le signal `gen_clken` du VTC est relié au `vtg_ce` du video-out — ce lien était correct depuis le début, ce n'était pas la cause du problème.

4. Débogage de la capture caméra

Une fois l'affichage validé avec une mire statique, le passage au flux caméra live a révélé un second problème : la caméra ne délivrait aucune image (`cap.read()` renvoyait toujours `False`, FOURCC vide, taille 0×0).

4.1 Diagnostic

L'outil `v4l2-ctl --list-formats-ext` a confirmé que la C270 supporte bien le 1280×720 en MJPG à 30 fps (le YUYV plafonne à 7,5 fps en 720p). Deux nœuds vidéo existent : `/dev/video0` (la caméra) et `/dev/video1` (metadata, aucun format).

4.2 Solution

La combinaison qui a débloqué la capture :

- Ouvrir la caméra par **chemin explicite** `"/dev/video0"` plutôt que par l'index 0.
- Ajouter un `time.sleep(0.5)` après l'ouverture, puis jeter une dizaine de frames (warm-up) le temps que le flux MJPG démarre.
- Forcer le FOURCC **MJPG avant** de fixer la résolution (sinon la caméra reste en YUYV plafonné).

5. Décalage horizontal résiduel

Le flux live s'affiche, mais avec un décalage horizontal circulaire fixe d'environ 256–288 pixels : le bord droit de l'image réapparaît sur la gauche. Le décalage étant constant et reproductible (et non aléatoire), il

s'agit d'un offset structurel d'alignement de ligne entre le VDMA et le VTC, et non d'un problème de phase au démarrage.

Une compensation logicielle simple est appliquée en attendant une correction matérielle propre : décaler le framebuffer du même nombre de pixels en sens inverse via `np.roll`.

```
rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
fb[:] = np.roll(rgb, -OFF, axis=1) # OFF ~ 256-288 px, à caler
fb.flush()
```

La mire graduée (`measure_offset.py`) permet de mesurer l'offset au pixel près : une graduation blanche tous les 128 px et un repère rouge au pixel 0. Pour une correction définitive, il faudra vérifier dans le block design les paramètres de timing horizontal du `v_tc_0` (Active 1280, Total 1650, HSync) et le mode du `v_axi4s_vid_out`, puis régénérer le bitstream.

6. Programmation du VDMA (canal MM2S)

Le canal MM2S (Memory-Map to Stream) lit le framebuffer en DDR et le pousse vers le video-out. Offsets de registres utilisés :

Registre	Offset	Rôle
MM2S_DMACR	0x00	Contrôle (reset = 0x4, run = 0x1)
MM2S_DMASR	0x04	Statut (bits d'erreur en [6:4])
MM2S_VSIZE	0x50	Nombre de lignes (l'écriture déclenche le transfert)
MM2S_HSIZE	0x54	Octets par ligne (WIDTH × BPP)
MM2S_FRMDLY_STRIDE	0x58	Stride (octets par ligne)
MM2S_START_ADDR	0x5C	Adresse frame 0 (puis +4, +8 pour frames 1, 2)

Séquence de démarrage : écrire DMACR=0x4 (reset), attendre, écrire DMACR=0x1 (run), renseigner les 3 slots d'adresse (ici la même adresse physique), puis le stride, le HSIZE et enfin le VSIZE. L'écriture du VSIZE déclenche le transfert. Le statut attendu est ~0x00010000 (aucun bit d'erreur).

Règle de sécurité : toujours remettre le VDMA en reset (DMACR=0x4) dans un bloc finally, sinon un relancement provoque l'erreur « DMA already running ».

7. Structure du dépôt Git

```
.
|- README.md
|- .gitignore
|- docs/
|   |- PA2_camera_hdmi_documentation.pdf (ce document)
|   |- images/ (captures, photos, mires)
|- hardware/
|   |- vivado_sources/ (design_1.v, wrapper, contraintes .xdc)
|   |- bitstream/ (design_1.bit + design_1.hwh)
|   |- README.md (notes de build Vivado)
|- src/
|   |- live_camera_hdmi.py (script principal caméra -> HDMI)
|   |- test_colorbar.py (mire statique, validation chemin HDMI)
|   |- test_camera_capture.py (capture caméra isolée, debug V4L2)
|   |- measure_offset.py (mire graduée pour mesurer le décalage)
|- notebooks/
|   |- camera_hdmi.ipynb (notebook optionnel)
```

8. État d'avancement

Étape	État
Chemin HDMI validé (mire statique)	Fait
Fix horloge pixel 74,25 MHz	Fait
Capture caméra C270 720p (V4L2/MJPEG)	Fait
Flux caméra live -> HDMI	Fait
Décalage horizontal (~256-288 px)	Compensation logicielle, fix HW à venir
Insertion IP MobileNetV2 (détection personne)	À venir

9. Contraintes & rappels matériels

- Alimenter la PYNQ-Z1 par l'alim **12V barrel**, pas par USB.
- La C270 ne délivre du 720p qu'en **MJPEG**.
- Toujours arrêter/reset le VDMA dans un bloc `finally`.
- Garder le projet hors d'un dossier synchronisé (OneDrive) : chemin C : /work/cam_hdmi.
- Le 720p est préférable au 1080p sur ce Zynq -1 (le 1080p exige 148,5 MHz pixel / 742,5 MHz TMDS, proche de la limite OSERDES). MobileNetV2 n'a pas besoin du 1080p.